



*Write code from scratch in a
clear & concise way, with a
complete basic course.*

*From beginners to
intermediate.*

Python for Beginners with applications

Lesson 6:

Error & Exception handling

NumPy library

Pandas library

Data visualization

Abdesselam Filali 2025-05-02 18h00 Algiers time

1.

Exceptions and Error handling



ZeroDivisionError is raised when a program attempts to perform a division operation where the denominator is zero.

Syntax: `ZeroDivisionError: division by zero`

Why :

Direct Division by Zero

Code:  3_01_exceptionHandling.py > ...

```
1  numerator = 10
2  denominator = 0
3  result = numerator / denominator
4  print(result)
```

Why : Variable Initialization Issue**Code:**

```
3_01_exceptionHandling.py > [denominator]
1 denominator = 0
2 result = 20 / denominator
```

Why : Conditional Statements Issue**Code:**

```
3_01_exceptionHandling.py > ...
1 denominator = 0
2 if denominator != 0:
3     result = 25 / denominator
```

Approach/Reason to Solve Zerodivisionerror

Use if-else

Code:  3_01_exceptionHandlingSol.py > ...

```
1   numerator = 10
2   denominator = 0
3
4   if denominator != 0:
5       |   result = numerator / denominator
6   else:
7       |   print("Error: Cannot divide by zero")
```

Using Conditional Expression

1 3_01_exceptionHandlingSol.py > ...

2 1 numerator = 10

3 2 denominator = 0

4 3

4 4 result = numerator / denominator if denominator != 0 else "Error: Denominator cannot be zero"

5 5 print(result)

Catching the Exception (Using Try-Except Block)

8 3_01_exceptionHandlingSol.py > ...

9 1 numerator = 10

10 2 denominator = 0

11 3 try:

12 4 | result = numerator / denominator

13 5 except ZeroDivisionError:

14 6 | print("Error: Cannot divide by zero")

Input a valid number

3_01_exceptionHandlingSolExample.py > ...

```
1  while True:
2      try:
3          x = int(input("Please enter a number: "))
4          break
5      except ValueError:
6          print("Oops! That was no valid number. Try again...")
7  print(x)
```

Sometimes it is necessary to raise an exception deliberately if a condition occurs that doesn't allow your program to proceed. You can raise exceptions using the statement.

```
course6 > 1_03_RaisingAnException.py > ...
1  def check_age(age):
2      if age < 0:
3          raise ValueError("Age cannot be negative.")
4      elif age < 18:
5          print("You are not old enough.")
6      else:
7          print("You are welcome.")
9  try:
10     user_age = int(input("Enter your age: "))
11     check_age(user_age)
12 except ValueError as e:
13     print("Error:", e)
```

By raising an exception, you can enforce certain conditions within your programs. It's particularly useful in data validation, where you need to ensure that input data conforms to expected parameters.

1 You can also define your own exceptions by extending the class. This
2 is useful when you need to create custom error messages for specific
3 business logic in your application. (Class, OOP)
4
5
6
7
8
9
10
11
12
13
14

Code:

```
3_02_errorHandling.py > ...  
1  # bad practice  
2  def read_file(file_path):  
3      file = open(file_path, 'r')  
4      content = file.read()  
5      file.close()  
6      return content  
7  print(read_file('textError.txt'))  
8  
9  # good practice  
10 def read_file(file_path):  
11     try:  
12         with open(file_path, 'r') as file:  
13             return file.read()  
14     except FileNotFoundError:  
15         return "File not found."  
16 print(read_file('text_Error.txt'))
```

2.

NumPy Library



BREAKING NEWS



1 What is NumPy?

2 NumPy (Numerical Python) is a fundamental powerful library for
3 mathematical and numerical operations.

4 It provides support for large, multi-dimensional arrays and matrices,
5 along with a wide range of mathematical functions to operate on these
6 arrays efficiently.

7 Why use NumPy?

- 8 - Much faster than Python lists for numerical tasks
- 9 - Used as the foundation for libraries like Pandas, TensorFlow, and Scikit-learn
- 9 - Makes matrix operations, statistics, and data transformations easy and efficient

10 installation

11 pip install numpy



<code>A[0,0], A[0,1], A[0,2]</code>	1	2	3
<code>A[1,0], A[1,1], A[1,2]</code>	5	6	7
<code>A[2,0], A[2,1], A[2,2]</code>	8	9	10
<code>A[3,0], A[3,1], A[3,2]</code>	11	12	13

What is NumPy Array?

A NumPy array is a powerful data structure that stores elements of the same data type in a grid-like format. It's much faster and more efficient than Python lists for numerical operations..

Creating Arrays:

```
import numpy as np

np.array([1, 2, 3])           # From a list
np.zeros((2, 3))              # Array of zeros
np.ones((3, 3))               # Array of ones
np.arange(0, 10, 2)           # Range of numbers
```

Creating Array:**Code:**

```
4_01_numpy.py > ...
1  import numpy as np
2  arr = np.array([1, 2, 3, 4, 5]) # Creating a NumPy array
3  result = arr * 2 # Performing mathematical
4  |         |         |         |         # operations on the array
5  print(result)      #output [2 4 6 8 10]
```


Basic Array Operations:**Indexing and slicing:**

```
arr = np.array([10, 20, 30, 40])  
arr[1]          # 20  
arr[1:3]        # [20 30]
```

Array Shape & Dimensions:

```
arr.shape        # Returns (rows, columns)  
arr.ndim         # Returns number of dimensions  
arr.size         # Total number of elements
```

Reshaping Arrays:

```
arr = np.array([[1, 2], [3, 4], [5, 6]])  
arr.reshape(2, 3) # Change the shape
```

Reshaping Arrays:

```
arr.dtype        # Get the data type of elements
```

Mathematical Operations:**Element-Wise operations:**

```
a = np.array([1, 2, 3])  
b = np.array([4, 5, 6])
```

```
a + b      # [5 7 9]  
a * b      # [4 10 18]  
a ** 2     # [1 4 9]
```

Aggregate functions:

```
arr = np.array([5, 10, 15])
```

```
np.sum(arr)      # 30  
np.mean(arr)     # 10.0  
np.max(arr)      # 15  
np.std(arr)      # Standard Deviation
```

Mathematical Operations:**Element-Wise operations:**

```
a = np.array([1, 2, 3])  
b = np.array([4, 5, 6])
```

```
a + b      # [5 7 9]  
a * b      # [4 10 18]  
a ** 2     # [1 4 9]
```

Aggregate functions:

```
arr = np.array([5, 10, 15])  
  
np.sum(arr)      # 30  
np.mean(arr)     # 10.0  
np.max(arr)      # 15  
np.std(arr)      # Standard Deviation
```

NumPy makes math easy – no need for loops! Just write it once and it works on the entire array.

Let's use NumPy in a real-world scenario: analyzing temperature data!
Example: Calculate Average Weekly Temperature

```
4_05_CalculateAvgWeeklyTemp.py > ...  
1  import numpy as np  
2  
3  # temperature in degrees celsius for 7 days  
4  temps=np.array([29.3, 42.1, 18.8, 22.1, 17.6, 28.7, 33.4])  
5  
6  #average temperature  
7  avg_temp=np.mean(temps)  
8  print(f"Average temperature for 7 days: {avg_temp:.1f} degrees celsius")  
9  
10 #highest temperature  
11 max_temp=np.max(temps)  
12 print(f"Highest temperature for 7 days: {max_temp:.1f} degrees celsius")  
13  
14 #lowest temperature  
15 min_temp=np.min(temps)  
16 print(f"Lowest temperature for 7 days: {min_temp:.1f} degrees celsius")
```

#1 **np.array()**: Create a NumPy array from a Python list or tuple.

#2 **np.zeros()**: Create an array filled with zeros of a specified shape.

#3 **np.ones()**: Create an array filled with ones of a specified shape.

#4 **np.arange()**: Create an array with values within a specified range.

#5 **np.linspace()**: Create an array with evenly spaced values over a specified interval.

#6 **np.reshape()**: Reshape an array into a new shape.

#7 **np.random.rand()**: The rand() function is used to generate an array with random values between 0 to 1.

#8 **np.random.randn()**: Generate an array of random numbers from a standard normal distribution. (close to zero)

#9 **np.random.randint()**: Generate an array of random integers within a specified range.

#10 **np.mean()**: Calculate the mean of array elements.

#11 **np.median()**: Calculate the median of array elements.

- #12 **np.std()**: Calculate the standard deviation of array elements.
- #13 **np.sum()**: Compute the sum of array elements.
- #14 **np.min(), np.max()**: Find the minimum and maximum values in an array.
- #15 **np.argmin(), np.argmax()**: Find the indices of the minimum and maximum values in an array.
- #16 **np.dot()**: Compute the dot product of two arrays.
- #17 **np.transpose()**: Calculate transpose of the array.
- #18 **np.concatenate()**: Concatenate arrays along a specified axis.

#19 **np.split()**: Split an array into multiple sub-arrays.

#20 **np.vstack()**, **np.hstack()**: Stack arrays vertically and horizontally.

#21 **np.unique()**: Find the unique elements in an array.

#22 **np.save()**, **np.load()**: Save and load arrays to/from disk.

#23 **np.clip()**: Clip (limit) the values in an array.

#24 **np.where()**: Return elements chosen from two arrays based on a condition.

#25 **np.linalg.inv()**: Calculate inverse of the matrix.

- #26 **np.linalg.det()**: Calculate determinant of the matrix.
- #27 **np.linalg.solve()**: Solve a system of linear equations.
- #28 **np.percentile()**: Compute the nth percentile of the data.
- #29 **np.corrcoef()**: Compute the correlation coefficient between two arrays.
- #30 **np.deg2rad()**, **np.rad2deg()**: Convert angles from degrees to radians and vice versa.
- #31 **np.argsort()**: Return the indices that would sort an array.
- #32 **np.searchsorted()**: Find indices where elements should be inserted to maintain order.

#33 `np.arcsin()`, `np.arccos()`, `np.arctan()`:
Inverse trigonometric functions.

#34 `np.linalg.eig()`: Eigenvalues and
eigenvectors of a square matrix.

#35 `np.linalg.svd()`: Singular value
decomposition.

3.

Pandas library



1 **What is Pandas?**

2 pandas is a fast, powerful, flexible and easy to use open source data
3 analysis and manipulation tool,
4 built on top of the Python programming language.

5 **Why use pandas?**

6 Pandas allows us to analyze big data and make conclusions based on
7 statistical theories.
8 Pandas can clean messy data sets, and make them readable and relevant.
9 Relevant data is very important in data science.

10 **installation**

11 pip install pandas

12

13

14



<https://www.w3schools.com/python/pandas/default.asp>

<https://pandas.pydata.org/>

What is Pandas?

pandas is a fast, powerful, flexible and easy to use open source data analysis and manipulation tool, built on top of the Python programming language.

Why use pandas?

Pandas allows us to analyze big data and make conclusions based on statistical theories.
Pandas can clean messy data sets, and make them readable and relevant.
Relevant data is very important in data science.

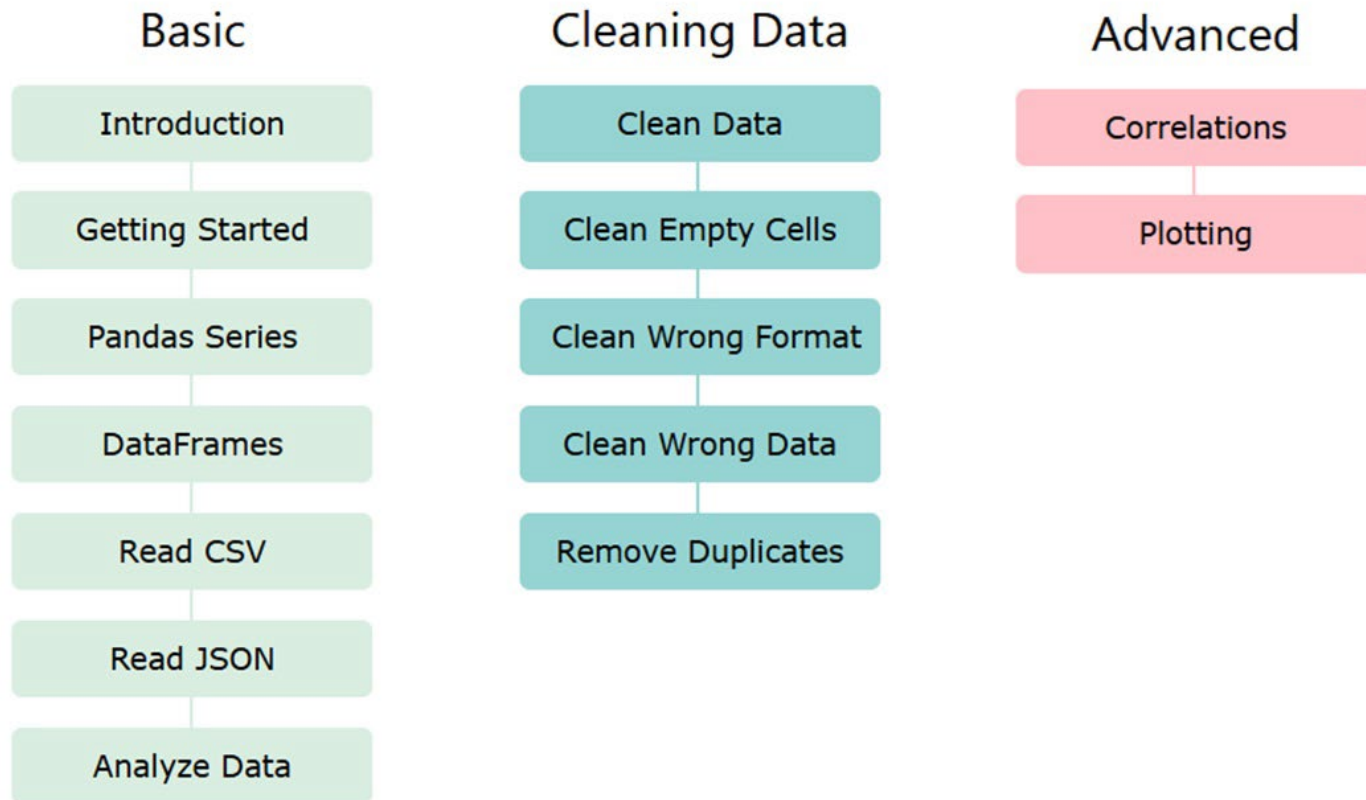
installation

```
pip install pandas
```



<https://www.w3schools.com/python/pandas/default.asp>
<https://pandas.pydata.org/>

1
2
3
4
5
6
7
8
9
10
11
12
13
14



4.

Data Visualisation



Matplotlib: The foundational plotting library, used for basic charts.

Seaborn: Built on top of Matplotlib, ideal for statistical plots.

Plotly: Used for interactive and web-based visualizations.

The Matplotlib logo features the word "matplotlib" in a blue sans-serif font. The "o" in "plot" is replaced by a circular icon containing a stylized multi-colored pie chart.

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html

The Seaborn logo consists of a circular icon on the left depicting a blue landscape with white mountains and a bar chart. To the right of the icon, the word "seaborn" is written in a blue sans-serif font.

<https://seaborn.pydata.org/>

The Plotly logo features a series of five vertical bars of increasing height on the left, each topped with a colored dot (pink, purple, blue, light blue, and light blue from left to right). To the right of the bars, the word "plotly" is written in a black sans-serif font.

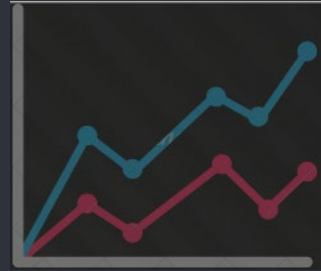
<https://plotly.com/python/>



Matplotlib is a Python 2D plotting library that produces high-quality figures in a variety of hard-copy formats and interactive environments across platforms. In Matplotlib, you can add features one by one, such as adding a title, labels, legends, and more.

Line plots

Library : matplotlib



Use Case: Showing trends over time.

In inline plotting, you should determine the x- and y-axes, and then you can add more features such as a title, a legend, and more.

Use Case: Showing trends over time.

```
course6 > 4_01_linePlot.py > ...
```

```
1  import matplotlib.pyplot as plt
2  x = [ '1', 2, 3, 4, 5]
3  z = [10, 12, 15, 17, 20]
4
5  plt.plot(x, z, 'bo')      # Plot x and y with circle markers on each point
6  plt.title('Line Plot Example') # Add a title to the plot
7  plt.xlabel('months')      # Label for the X-axis
8  plt.ylabel('salary')      # Label for the Y-axis
9  plt.grid(which = "both")
10 plt.show()                # Display the plot
```



7th > 4_01_linePlot1.py > ...

```
1  import matplotlib.pyplot as plt
2
3  year = ['2015', 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023]
4  capitalization = [38.76, 39.48, 52.48, 52.06, 66.73, 87.31, 144.33, 87.91, 118.36]
5
6  plt.plot(year, capitalization, marker='o')
7  plt.title('Google Capitalization for the past 9 years')
8  plt.xlabel('year')
9  plt.ylabel('Market Capitalization')
10 plt.grid(which = 'both')
11 plt.show()
```

12

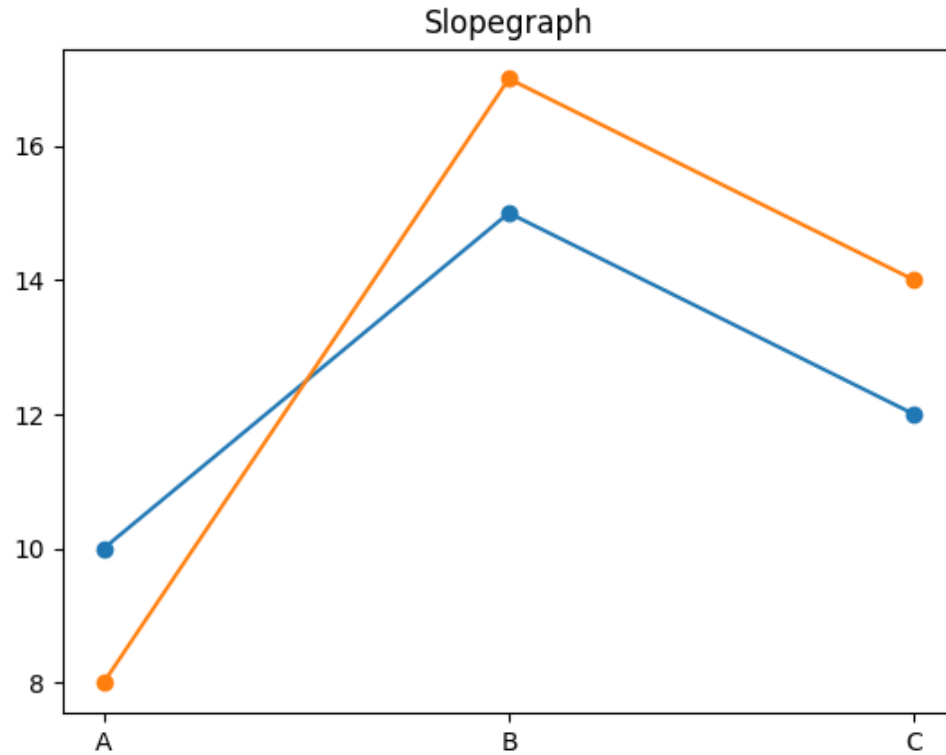
13

14



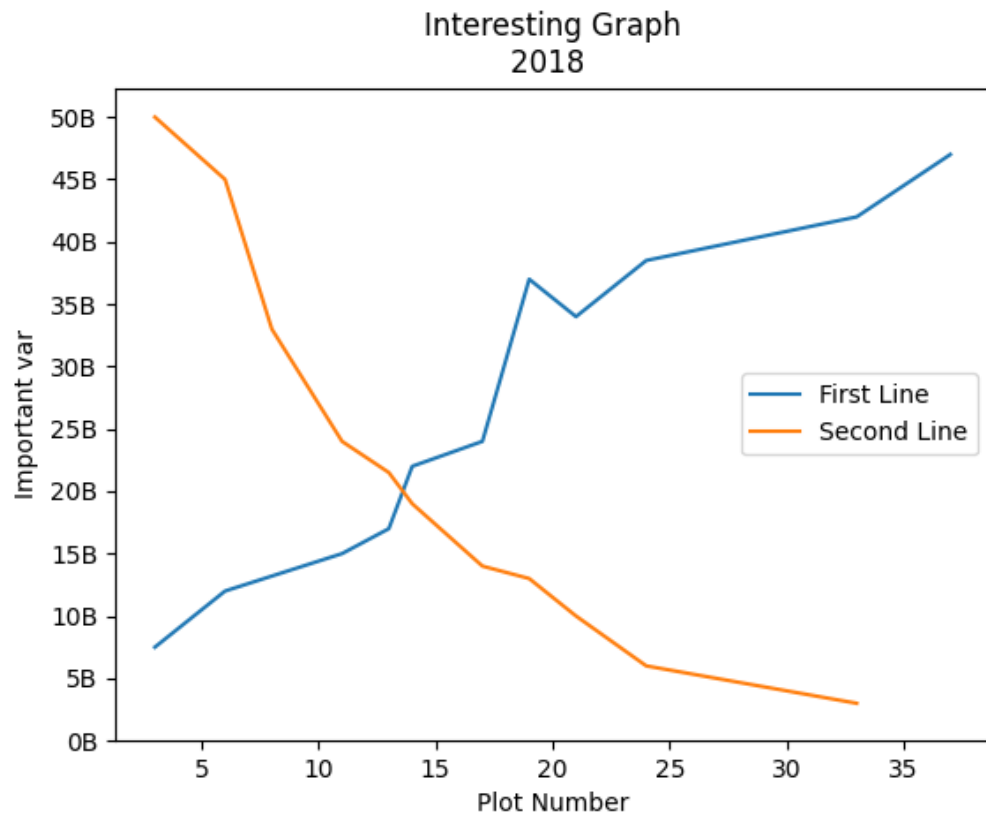
```
7th > 4_01_linePlot2.py > ...  
1  import matplotlib.pyplot as plt  
2  categories = ['A', 'B', 'C']  
3  values_1 = [10, 15, 12]  
4  values_2 = [8, 17, 14]  
5  plt.plot(categories, values_1, marker='o', label='2023')  
6  plt.plot(categories, values_2, marker='o', label='2024')  
7  plt.title('Slopegraph')  
8  plt.show()
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14



7th > 4_01_linePlot3.py > ...

```
1  import matplotlib.pyplot as plt
2  x =[3, 6 , 8, 11, 13, 14, 17, 19, 21, 24, 33, 37]
3  y =[7.5, 12, 13.2, 15, 17, 22, 24, 37, 34, 38.5, 42,47]
4
5  x2 =[3, 6, 8, 11, 13, 14, 17, 19, 21, 24, 33]
6  y2 =[50, 45, 33, 24, 21.5, 19, 14, 13, 10, 6, 3]
7  plt.plot(x, y, label='First Line' )
8  plt.plot(x2, y2, label='Second Line' )
9  plt.xlabel('Plot Number' )
10 plt.ylabel('Important var' )
11 plt.title('Interesting Graph\n2018 ')
12 plt.yticks([0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50],
13            ['0B', '5B', '10B', '15B', '20B', '25B', '30B', '35B', '40B' , '45B', '50B'])
14 plt.legend()
15 plt.show()
```





Use Case: Comparing different categories or discrete items.

course6 > 4_02_BarPlot1.py > ...

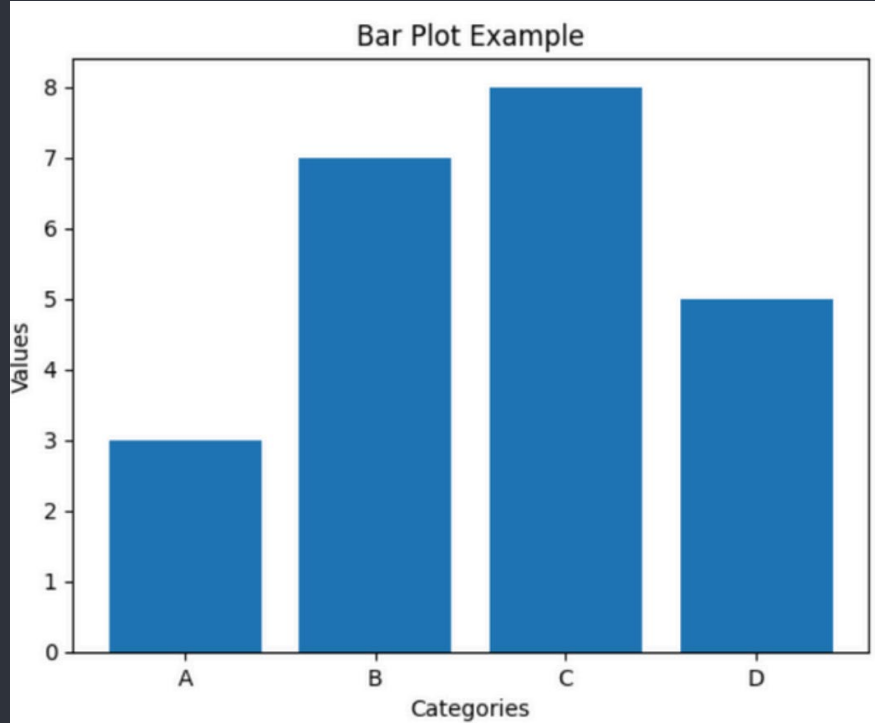
```
1 import matplotlib.pyplot as plt # Import the Matplotlib library
2 # Data
3 categories = ['A', 'B', 'C', 'D'] # Define categories
4 values = [3, 7, 8, 5] # Corresponding values for each category
5 # Create Bar Plot
6 plt.bar(categories, values) # Create a bar plot using categories and values
7 plt.title('Bar Plot Example') # Add a title to the plot
8 plt.xlabel('Categories') # Label for the X-axis
9 plt.ylabel('Values') # Label for the Y-axis
10 plt.show() # Display the plot
```

Bar plots

Library : matplotlib



Use Case: Comparing different categories or discrete items.



Bar plots

Library : matplotlib



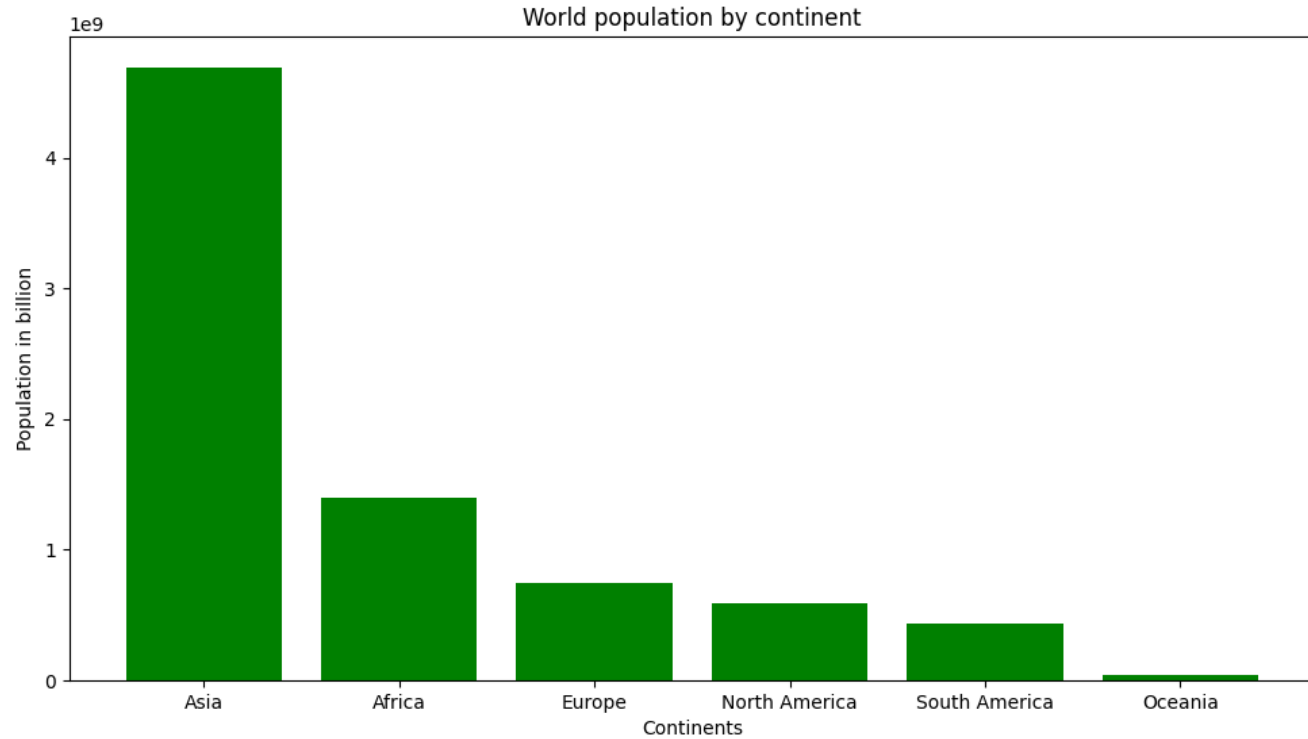
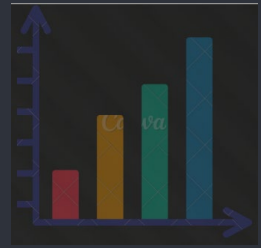
Use Case: Comparing different categories or discrete items.

```
course6 > 4_02_BarPlot.py > ...
```

```
1 import matplotlib.pyplot as plt
2 continents=['Asia', 'Africa', 'Europe', 'North America', 'South America', 'Oceania']
3 population= [4694576167, 1393676444, 745173774, 595783465, 434254119, 44491724]
4 plt.bar(continents, population, color='green')
5 plt.title('World population by continent')
6 plt.xlabel('Continents')
7 plt.ylabel('Population in billion')
8 plt.show()
```

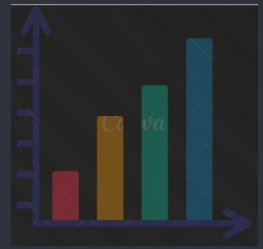
Bar plots

Library : matplotlib



Horizontal Bar plots

Library : matplotlib



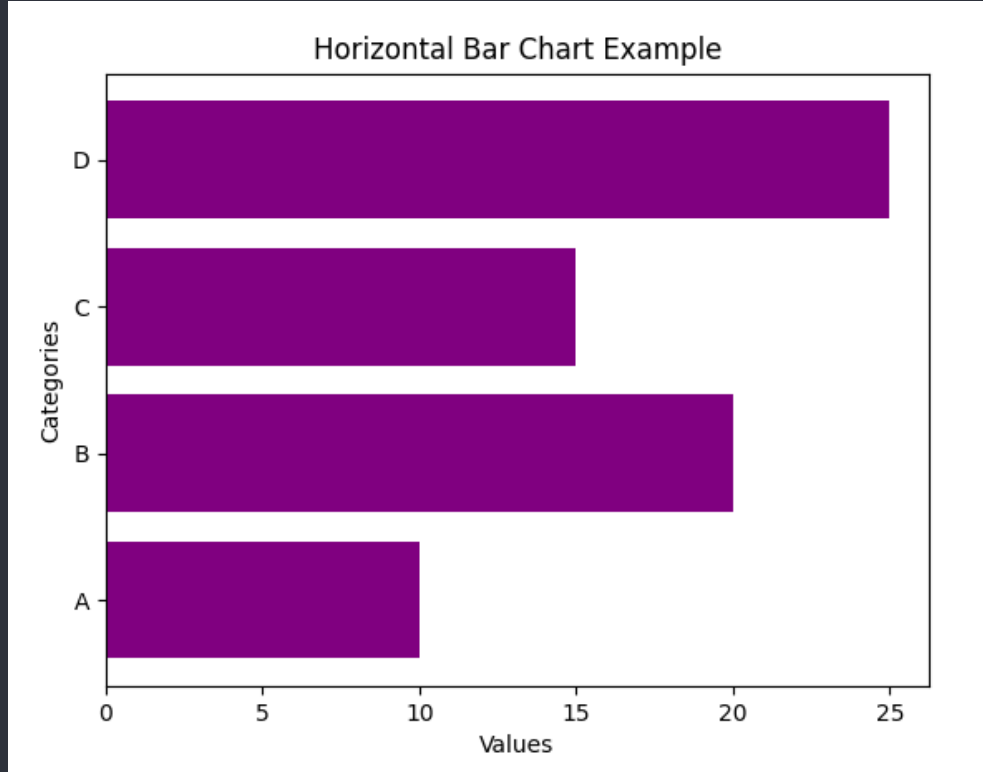
Use Case: Comparing different categories or discrete items.

```
course6 > 4_03_barh.py > ...
```

```
1  import matplotlib.pyplot as plt
2
3  categories = ['A', 'B', 'C', 'D']
4  values = [10, 20, 15, 25]
5
6  plt.barh(categories, values, color='purple')
7
8  plt.xlabel("Values")
9  plt.ylabel("Categories")
10 plt.title("Horizontal Bar Chart Example")
11
12 plt.show()
```

Horizontal Bar plots

Library : matplotlib



Scatter plots

Library : matplotlib



Use Case: Showing relation between 2 variables.

```
course6 > 4_04_ScatterPlot.py > ...
```

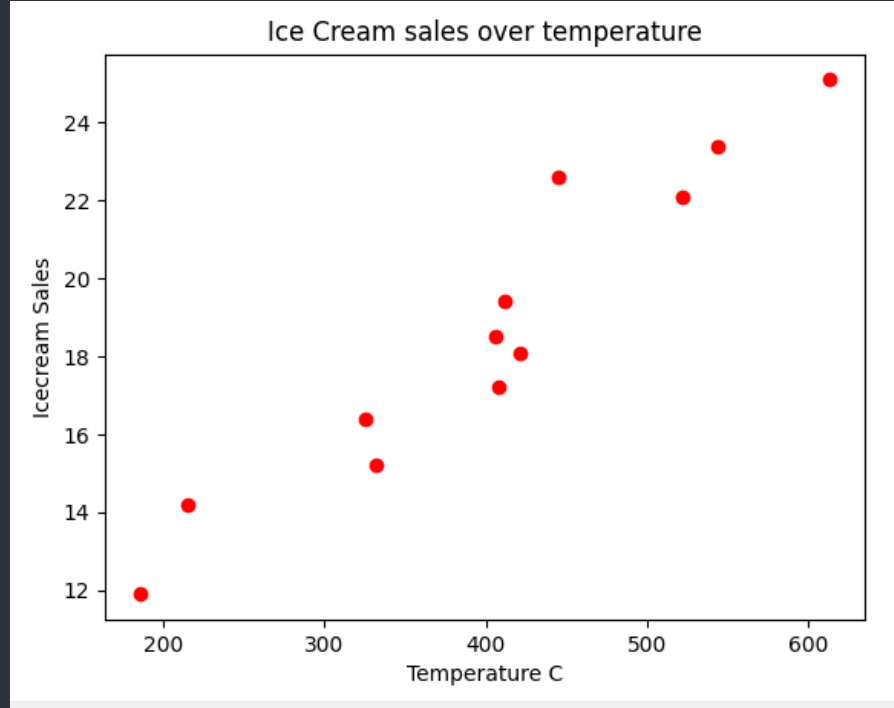
```
1  import matplotlib.pyplot as plt
2  iceCreamSales = [215, 325, 185, 332, 406, 522, 412, 614, 544, 421, 445, 408]
3  temperature = [14.2, 16.4, 11.9, 15.2, 18.5, 22.1, 19.4, 25.1, 23.4, 18.1, 22.6, 17.2 ]
4  plt.scatter(iceCreamSales, temperature, color='red')
5  plt.title('Ice Cream sales over temperature')
6  plt.xlabel('Temperature C')
7  plt.ylabel('Icecream Sales')
8  plt.show()
```

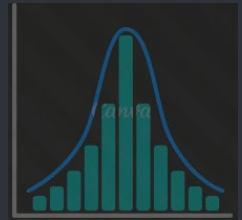
Scatter plots

Library : matplotlib



Use Case: Showing relation between 2 variables.





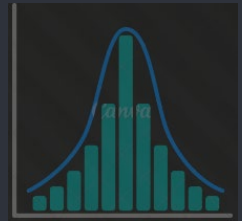
Use Case: Showing the distribution of data.

```
course6 > 4_06_Histogram.py > ...
```

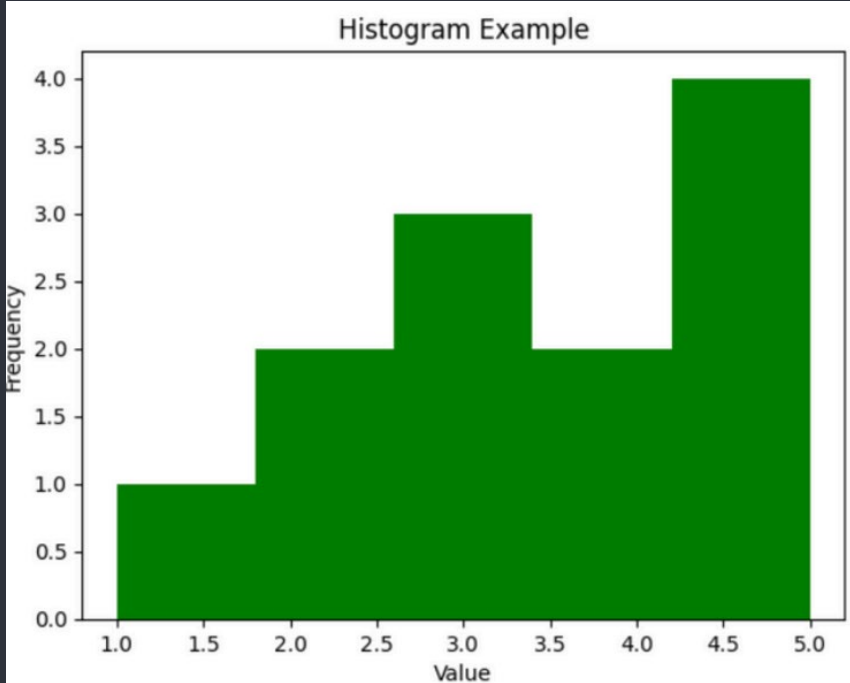
```
1 import matplotlib.pyplot as plt
2 grades = [1, 2, 2, 3, 3, 3, 4, 4, 5, 5, 5, 5]
3 plt.hist(grades, bins=5, color='green')
4 plt.title('Histogram Example')
5 plt.xlabel('Value')
6 plt.ylabel('Frequency')
7 plt.show()
```

Histogram plots

Library : matplotlib



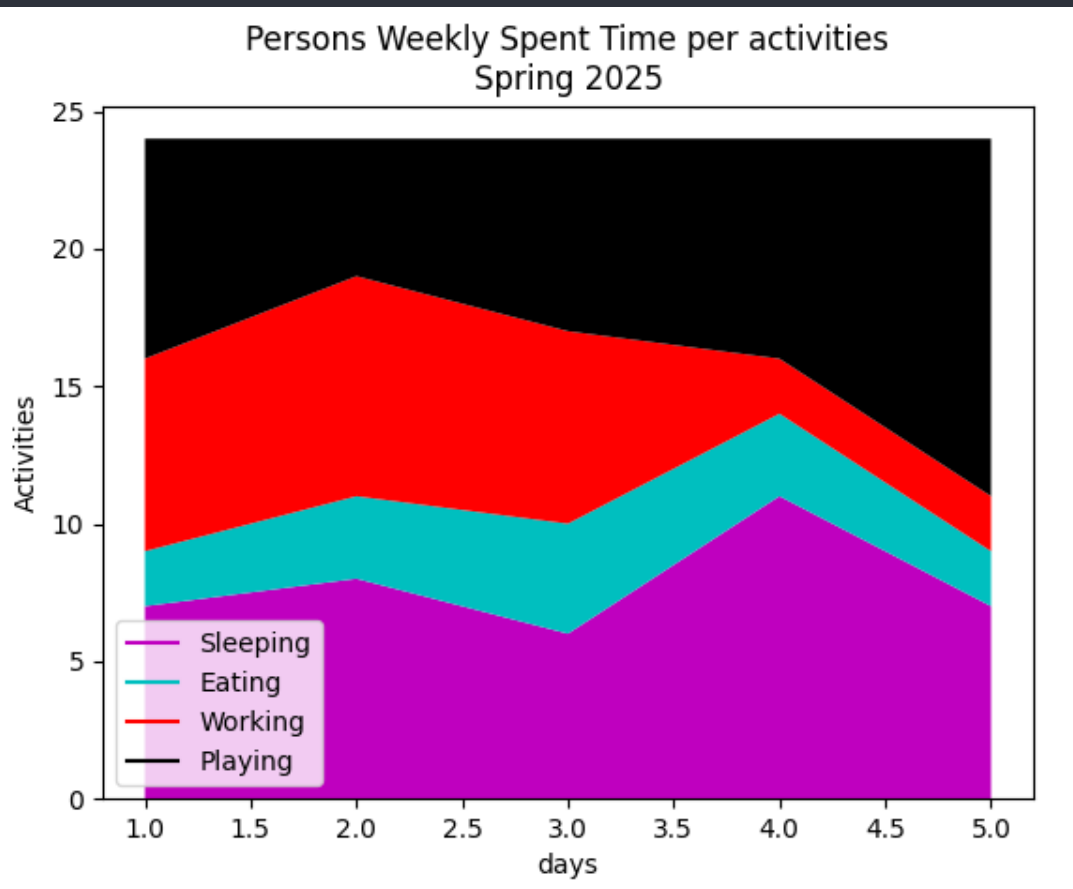
Use Case: Showing the distribution of data.



Use Case: Stack plots present the frequency of every activity, such as the frequency of sleeping, eating, working, and playing per day. In this data set, on day 2, a person spent eight hours sleeping, three hours in eating, eight hours working, and five hours playing.

```
course6 > 4_08_Stack plot.py > ...
```

```
1  import matplotlib.pyplot as plt
2  days = [1, 2, 3, 4, 5]
3  sleeping = [7, 8, 6, 11, 7]
4  eating = [2, 3, 4, 3, 2]
5  working = [7, 8, 7, 2, 2]
6  playing = [8, 5, 7, 8, 13]
7  plt.plot([],[], color='m', label='Sleeping')
8  plt.plot([],[], color='c', label='Eating')
9  plt.plot([],[], color='r', label='Working')
10 plt.plot([],[], color='k', label='Playing')
11 plt.stackplot(days, sleeping, eating, working ,
12              playing, colors=['m','c', 'r', 'k'])
13 plt.xlabel('days')
14 plt.ylabel('Activities')
15 plt.title('Persons Weekly Spent Time per activities\nSpring 2025')
16 plt.legend()
17 plt.show()
```

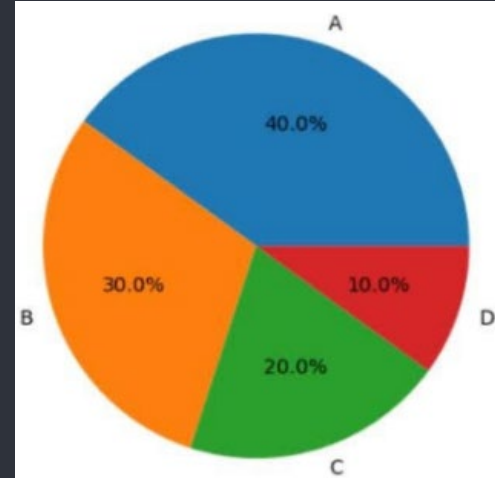




Use Case: Showing the distribution of data.

```
course6 > 4_06_Pie.py > ...
```

```
1 import matplotlib.pyplot as plt
2 sizes = [40, 30, 20, 10]
3 labels = ['A', 'B', 'C', 'D']
4 plt.pie(sizes, labels=labels, autopct='%1.1f%%')
5 plt.show()
```



Donut chart

Library : matplotlib



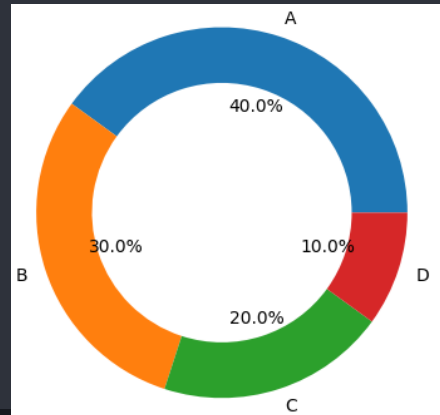
Use Case: Showing the distribution of data.

```
course6 > 4_07_Donut.py > ...
```

```
1 import matplotlib.pyplot as plt
2 sizes = [40, 30, 20, 10]
3 labels = ['A', 'B', 'C', 'D']
4 plt.pie(sizes, labels=labels, autopct='%1.1f%%', wedgeprops={'width':0.3})
5 plt.show()
```

<https://python-graph-gallery.com/donut-plot/>

https://matplotlib.org/stable/gallery/pie_and_polar_charts/pie_and_donut_labels.html





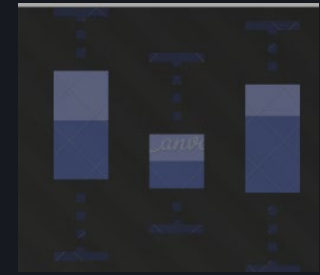
Use Case: How to customize Pie charts using matplotlib.

```
course6 > 4_06_PieCustomize1.py > ...
```

```
1  age_group_labels = [  
2      'Children 0-18', 'Adults 19-25', 'Adults 26-34',  
3      'Adults 35-54', 'Adults 55-64', 'Seniors 65+'  
4  ]  
5  age_group_populations = [  
6      75307800, 27799100, 39817700,  
7      81478600, 42061700, 52784400  
8  ]  
9  # print age groups and corresponding populations  
10 for label, population in zip(age_group_labels, age_group_populations):  
11     print (f'{label}:    {population:,}')
```

Use Case: Summarizing the distribution of a dataset, showing quartiles and outliers.

BOX plot



```
import seaborn as sns # Import the Seaborn library
import matplotlib.pyplot as plt # Import Matplotlib for additional control

# Data
data = [10, 12, 15, 13, 17, 19, 10, 9] # Sample data for the box plot

# Create Box Plot
sns.boxplot(data=data) # Create a box plot to show distribution, quartiles, and outliers
plt.title('Box Plot Example') # Add a title to the plot
plt.show() # Display the plot
```

BOX PLOT example

